

# Traitement des Données Multimodales

APPRENDRE À CODER AVEC PYTHON

13 août 2026

Prof. Abdellah Adib

`abdellah.adib@fstm.ac.ma`





# Sommaire

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Vous savez déjà  
programmer

Variables et typage  
dynamique

Vérité, affichage,  
saisie

Avant-propos

Environnement  
Python

Outils de  
contrôle de flux

Les collections  
de données

Les fonctions

Les modules

Fichiers et  
entrées/sorties

1

## De C à Python : une transition

Vous savez déjà programmer

Variables et typage dynamique

Vérité, affichage, saisie

## Avant-propos

## Environnement Python

## Outils de contrôle de flux

## Les collections de données

## Les fonctions

## Les modules

## Fichiers et entrées/sorties

# Point de départ : vos acquis

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Vous savez déjà  
programmer

Variables et typage  
dynamique

Vérité, affichage,  
saisie

Avant-propos

Environnement  
Python

Outils de  
contrôle de flux

Les collections  
de données

Les fonctions

Les modules

Fichiers et  
entrées/sorties

2

- ◆ Vous maîtrisez déjà l'**algorithmique** et le **langage C** : variables, conditions, boucles, fonctions, tableaux.
- ◆ L'objectif n'est donc **pas** de réapprendre à programmer, mais de **transférer** ce savoir vers **Python**.
- ◆ Nous procédons **par contraste** : pour chaque notion, *ce qui est identique* au C et surtout *ce qui change*.
  - ✧ Les concepts communs (séquence, test, boucle) sont rappelés brièvement.
  - ✧ L'effort porte sur les **pièges spécifiques** à un développeur venant du C.
- ◆ Le temps ainsi dégagé est consacré à la maîtrise des fonctionnalités Python essentielles pour la manipulation des données (**tranches, compréhensions, NumPy**).

# Python en une phrase, pour un développeur C

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Vous savez déjà  
programmer

Variables et typage  
dynamique

Vérité, affichage,  
saisie

Avant-propos

Environnement  
Python

Outils de  
contrôle de flux

Les collections  
de données

Les fonctions

Les modules

Fichiers et  
entrées/sorties

3

- ◆ **Python** (Guido van Rossum, 1991) est un langage **interprété**, à **typage dynamique**, à **gestion mémoire automatique**.
- ◆ Concrètement, par rapport au C :
  - ✧ Pas de **déclaration** ni de **compilation** explicite.
  - ✧ Les **blocs** sont délimités par l'**indentation**, pas par des accolades.
  - ✧ Pas de gestion manuelle de la mémoire (`malloc/free`).
  - ✧ Une riche bibliothèque standard, plus l'écosystème scientifique (NumPy, Pandas, OpenCV, ...).
- ◆ Le prix à payer : la vitesse d'écriture se gagne contre une **vigilance sur les types** au moment de l'exécution.

# Correspondance C → Python (1/2)

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Vous savez déjà  
programmer

Variables et typage  
dynamique

Vérité, affichage,  
saisie

Avant-propos

Environnement

Python

Outils de

contrôle de flux

Les collections

de données

Les fonctions

Les modules

Fichiers et  
entrées/sorties

4

| Notion            | En C (acquis)                                  | En Python (le changement)                  |
|-------------------|------------------------------------------------|--------------------------------------------|
| Déclaration       | <code>int x = 3;</code> (type figé)            | <code>x = 3</code> : typage dynamique      |
| Fin d'instruction | <code>;</code> obligatoire                     | fin de ligne (pas de <code>;</code> )      |
| Blocs             | accolades <code>{ }</code>                     | indentation + <code>:</code>               |
| Boucle comptée    | <code>for(i=0;i&lt;n;i++)</code>               | <code>for i in range(n):</code> (for-each) |
| Tableau           | <code>int t[10]</code> : homogène, taille fixe | <code>list</code> : dynamique, hétérogène  |

# Correspondance C → Python (2/2)

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Vous savez déjà  
programmer

Variables et typage  
dynamique

Vérité, affichage,  
saisie

Avant-propos

Environnement

Python

Outils de  
contrôle de flux

Les collections  
de données

Les fonctions

Les modules

Fichiers et  
entrées/sorties

5

| Notion    | En C (acquis)                           | En Python (le changement)                   |
|-----------|-----------------------------------------|---------------------------------------------|
| Chaîne    | <code>char* + \0</code> , mutable       | <code>str</code> immuable, indices négatifs |
| Vérité    | 0 / non-0                               | vides et <code>None</code> sont faux        |
| Affichage | <code>printf("%d", x);</code>           | <code>print(f"{x}")</code>                  |
| Saisie    | <code>scanf("%d", &amp;x);</code>       | <code>x = int(input())</code>               |
| Mémoire   | <code>malloc</code> / <code>free</code> | ramasse-miettes (GC)                        |
| Exécution | compilation → binaire                   | interprétation ligne à ligne                |

# Le même programme, deux langages

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Vous savez déjà  
programmer

Variables et typage  
dynamique

Vérité, affichage,  
saisie

Avant-propos

Environnement  
Python

Outils de  
contrôle de flux

Les collections  
de données

Les fonctions

Les modules

Fichiers et  
entrées/sorties

6

## En C

```
#include <stdio.h>
int main(void) {
    int n = 5, somme = 0;
    for (int i = 1; i <= n; i++) {
        somme += i;
    }
    printf("Somme = %d\n", somme);
    return 0;
}
```

- ◆ Ni type déclaré,
- ◆ Ni accolades,
- ◆ Ni ;,
- ◆ Ni main : l'indentation et range portent toute la structure.

## En Python

```
n = 5
somme = 0
for i in range(1, n + 1):
    somme += i
print(f"Somme = {somme}")
```

Les deux affichent Somme = 15.



# Sommaire

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Vous savez déjà  
programmer

Variables et typage  
dynamique

Vérité, affichage,  
saisie

Avant-propos

Environnement  
Python

Outils de  
contrôle de flux

Les collections  
de données

Les fonctions

Les modules

Fichiers et  
entrées/sorties

De C à Python : une transition

Vous savez déjà programmer

Variables et typage dynamique

Vérité, affichage, saisie

Avant-propos

Environnement Python

Outils de contrôle de flux

Les collections de données

Les fonctions

Les modules

Fichiers et entrées/sorties

7



# Variables : pas de déclaration

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Vous savez déjà  
programmer

Variables et typage  
dynamique

Vérité, affichage,  
saisie

Avant-propos

Environnement  
Python

Outils de  
contrôle de flux

Les collections  
de données

Les fonctions

Les modules

Fichiers et  
entrées/sorties

8

♦ En **Python**, une variable n'est pas déclarée : elle existe dès la première affectation, et le type suit la **valeur**, pas la variable.

```
x = 3           # x reference un entier
type(x)         # <class 'int'>
x = "trois"     # legal : x reference maintenant une chaine
type(x)         # <class 'str'>
```

## Piège — vous venez du C

En C, le type est figé à la déclaration. En Python, réaffecter une variable à une valeur d'un autre type est autorisé : c'est au programmeur de suivre le type courant.

# Les types de base

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Vous savez déjà  
programmer

Variables et typage  
dynamique

Vérité, affichage,  
saisie

Avant-propos

Environnement  
Python

Outils de  
contrôle de flux

Les collections  
de données

Les fonctions

Les modules

Fichiers et  
entrées/sorties

9

♦ Six types de base suffisent pour débiter :

| Type     | Exemple                        | Commentaire                   |
|----------|--------------------------------|-------------------------------|
| bool     | <code>actif = True</code>      | valeurs True / False          |
| int      | <code>rayon = 6_371_000</code> | entiers à précision illimitée |
| float    | <code>pi = 3.14159</code>      | nombres à virgule flottante   |
| str      | <code>licence = "IRM"</code>   | texte (chaîne immuable)       |
| complex  | <code>onde = 3 + 6j</code>     | nombres complexes             |
| NoneType | <code>stock = None</code>      | absence de valeur             |

♦ Le type long du C n'existe pas : un int n'a pas de borne.

# Opérations arithmétiques : le piège de la division

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Vous savez déjà  
programmer

Variables et typage  
dynamique

Vérité, affichage,  
saisie

Avant-propos

Environnement  
Python

Outils de  
contrôle de flux

Les collections  
de données

Les fonctions

Les modules

Fichiers et  
entrées/sorties

10

♦ Les opérateurs sont proches du C, avec **une exception majeure** sur la division.

```
a = 3
a / 2      # division reelle -> 1.5    (en C, 3/2 vaut 1)
a // 2     # division entiere -> 1
7 % 3      # modulo                -> 1
2 ** 10    # puissance              -> 1024
```

## Piège — vous venez du C

En C, / entre deux entiers fait une division *entière*. En Python 3, / renvoie **toujours** un float ; utilisez // pour un quotient entier.

109



# Sommaire

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Vous savez déjà  
programmer  
Variables et typage  
dynamique

Vérité, affichage,  
saisie

Avant-propos

Environnement  
Python

Outils de  
contrôle de flux

Les collections  
de données

Les fonctions

Les modules

Fichiers et  
entrées/sorties

11

De C à Python : une transition

Vous savez déjà programmer

Variables et typage dynamique

Vérité, affichage, saisie

Avant-propos

Environnement Python

Outils de contrôle de flux

Les collections de données

Les fonctions

Les modules

Fichiers et entrées/sorties

109

# La vérité en Python (*truthiness*)

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Vous savez déjà  
programmer  
Variables et typage  
dynamique

Vérité, affichage,  
saisie

12

Avant-propos

Environnement  
Python

Outils de  
contrôle de flux

Les collections  
de données

Les fonctions

Les modules

Fichiers et  
entrées/sorties

♦ La condition d'un `if` accepte n'importe quel objet, converti en booléen.

```
bool(0)           # False
bool(0.0)         # False
bool("")          # False   chaîne vide
bool([])          # False   liste vide
bool(None)        # False
bool(42)          # True
bool("a")         # True
```

## Piège — vous venez du C

En C, seule la valeur 0 est fausse. En Python, sont **aussi** faux : la chaîne vide, les collections vides et None. Cela permet d'écrire `if ma_liste:` au lieu de `if len(ma_liste) > 0:`.

# Affichage : les *f-strings*

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Vous savez déjà  
programmer  
Variables et typage  
dynamique

Vérité, affichage,  
saisie

Avant-propos

Environnement  
Python

Outils de  
contrôle de flux

Les collections  
de données

Les fonctions

Les modules

Fichiers et  
entrées/sorties

13

♦ La manière moderne et recommandée de formater est la **f-string** (préfixe **f**).

```
nom = "Ahmed"
age = 32
print(f"{nom} a {age} ans")      # Ahmed a 32 ans
print(f"1/3 = {1/3:.3f}")       # 1/3 = 0.333
```

✧ Le format : `.3f` joue le rôle du `%.3f` de `printf`. L'ancienne écriture `"...".format(...)` reste valide mais est moins lisible.

# Saisie : input () renvoie toujours une chaîne

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Vous savez déjà  
programmer  
Variables et typage  
dynamique

Vérité, affichage,  
saisie

Avant-propos

Environnement  
Python

Outils de  
contrôle de flux

Les collections  
de données

Les fonctions

Les modules

Fichiers et  
entrées/sorties

14

♦ Contrairement à scanf, input () ne connaît pas le type attendu : il renvoie **toujours** un str.

```
val = input("Entrez un entier : ")
type(val)           # <class 'str'>

# int(val) renvoie une NOUVELLE valeur : il faut la reassigner
val = int(val)       # sinon val reste une chaîne
# ou, directement, en une seule ligne :
n = int(input("Entrez un entier : "))
```

## Piège — vous venez du C

int(val) ne modifie pas val sur place : il produit un entier qu'il faut **recupérer**. Sans réaffectation, un calcul sur val échoue ou concatène du texte.

109

# Synthèse : les pièges classiques du C-iste

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Vous savez déjà  
programmer  
Variables et typage  
dynamique

Vérité, affichage,  
saisie

Avant-propos

Environnement  
Python

Outils de  
contrôle de flux

Les collections  
de données

Les fonctions

Les modules

Fichiers et  
entrées/sorties

15

- ◆ Indentation à la place des accolades ; ne pas oublier le `:`.
- ◆ Pas d'opérateurs `++` / `--` : utiliser `+= 1`.
- ◆ `=` affecte, `==` compare (comme en C, mais le compilateur ne vous protège plus).
- ◆ `/` = division réelle ; `//` = division entière.
- ◆ `range(a, b)` exclut la borne `b`.
- ◆ `input()` renvoie une chaîne : convertir avant tout calcul.
- ◆ L'affectation copie une **référence**, pas la valeur : attention aux listes partagées.



# Pont vers le module M351

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Vous savez déjà  
programmer  
Variables et typage  
dynamique

Vérité, affichage,  
saisie

Avant-propos

Environnement  
Python

Outils de  
contrôle de flux

Les collections  
de données

Les fonctions

Les modules

Fichiers et  
entrées/sorties

16

- ◆ Ces fondations servent un objectif précis : manipuler des **données multimodales** (texte, image, audio).
- ◆ La liste Python prépare le **tableau NumPy** (ndarray), structure centrale du module : une image est un tableau de pixels, un signal audio un tableau d'échantillons.
- ◆ Les bonnes habitudes prises ici (types, conversion, *f-strings*) conditionnent la fiabilité de toute la chaîne de traitement à venir (Pandas, NLTK, OpenCV, Pydub).
- ◆ La suite du chapitre approfondit chaînes, collections, fonctions, modules et fichiers, dans le même esprit de transition.



# Sommaire

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Avant-propos  
Origine de Python  
Pourquoi Python

Environnement  
Python

Outils de  
contrôle de flux

Les collections  
de données

Les fonctions

Les modules

Fichiers et  
entrées/sorties

De C à Python : une transition

**Avant-propos**

Origine de Python

Pourquoi Python

Environnement Python

Outils de contrôle de flux

Les collections de données

Les fonctions

Les modules

Fichiers et entrées/sorties

17

109

# Origine de Python

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Avant-propos  
Origine de Python  
Pourquoi Python

Environnement  
Python

Outils de  
contrôle de flux

Les collections  
de données

Les fonctions

Les modules

Fichiers et  
entrées/sorties

18

- ◆ Le langage **Python** a été créé en 1989 par **Guido van Rossum** (Pays-Bas), avec de nombreux contributeurs bénévoles.



- ✧ Première version publiée le 20 février 1991.
- ✧ Le nom vient de la troupe britannique *Monty Python*, pas du reptile.
- ✧ Le langage évolue et la dernière version stable "3.14.7" date du 07 Octobre 2025 .
- ◆ **Python** est aujourd'hui l'un des langages les plus utilisés en science des données et en intelligence artificielle.



# Sommaire

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Avant-propos

Origine de Python

Pourquoi Python

Environnement  
Python

Outils de  
contrôle de flux

Les collections  
de données

Les fonctions

Les modules

Fichiers et  
entrées/sorties

De C à Python : une transition

Avant-propos

Origine de Python

Pourquoi Python

Environnement Python

Outils de contrôle de flux

Les collections de données

Les fonctions

Les modules

Fichiers et entrées/sorties

19

109

# Pourquoi Python ?

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Avant-propos  
Origine de Python  
Pourquoi Python

Environnement  
Python

Outils de  
contrôle de flux

Les collections  
de données

Les fonctions

Les modules

Fichiers et  
entrées/sorties

20

- ◆ **Python** est un langage portable, dynamique, extensible et gratuit, qui permet (sans l'imposer) une approche modulaire de la programmation.
- ◆ Ses caractéristiques principales :
  - ✧ **Multiplateforme** : Windows, macOS, Linux/Unix, Android, iOS, du Raspberry Pi au supercalculateur.
  - ✧ **Interprété** : un script n'a pas besoin d'être compilé, contrairement au C ou au C++.
  - ✧ **Gratuit** : installable sur autant de machines que souhaité.
  - ✧ **Riche** : bibliothèque standard étendue et écosystème de paquets contribués.



# Applications de Python

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Avant-propos  
Origine de Python  
Pourquoi Python

Environnement  
Python

Outils de  
contrôle de flux

Les collections  
de données

Les fonctions

Les modules

Fichiers et  
entrées/sorties

21

- ◆ **Data Science et analyse** : Pandas, NumPy, Matplotlib, Seaborn.
- ◆ **Apprentissage automatique et IA** : TensorFlow, Keras, PyTorch, Scikit-learn.
- ◆ **Traitement d'images et de vidéos** : OpenCV, Pillow, Scikit-Image.
- ◆ **Traitement des signaux audio** : Librosa, Soundfile, Pydub.
- ◆ **Développement Web** : Django, Flask, FastAPI.
- ◆ **Automatisation et scripts** : BeautifulSoup, Scrapy, Selenium.

109



# Sommaire

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Avant-propos

Environnement  
Python

Installer Python

Environnement et  
reproductibilité

Notion de variable

Types standards

Chaînes de caractères

Affichage

Outils de  
contrôle de flux

Les collections  
de données

Les fonctions

Les modules

Fichiers et  
entrées/sorties

22

De C à Python : une transition

Avant-propos

**Environnement Python**

Installer Python

Environnement et reproductibilité

Notion de variable

Types standards

Chaînes de caractères

Affichage

Outils de contrôle de flux

Les collections de données

Les fonctions

Les modules

Fichiers et entrées/sorties

109

# Installer et configurer Python

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition  
Avant-propos  
Environnement  
Python

Installer Python

Environnement et  
reproductibilité  
Notion de variable  
Types standards  
Chaînes de caractères  
Affichage

Outils de  
contrôle de flux

Les collections  
de données

Les fonctions

Les modules

Fichiers et  
entrées/sorties

23

- ♦ La maîtrise de **Python** passe par la pratique : il faut donc l'installer sur sa machine.
- ♦ Téléchargement depuis le site officiel :

<https://www.python.org/downloads/>

## Download the latest version for Windows

Download Python install manager

Or get the standalone installer for [Python 3.14.7](#)

Looking for Python with a different OS? Python for [Windows](#),  
[Linux/Unix](#), [macOS](#), [Android](#), [iOS](#), [other](#)

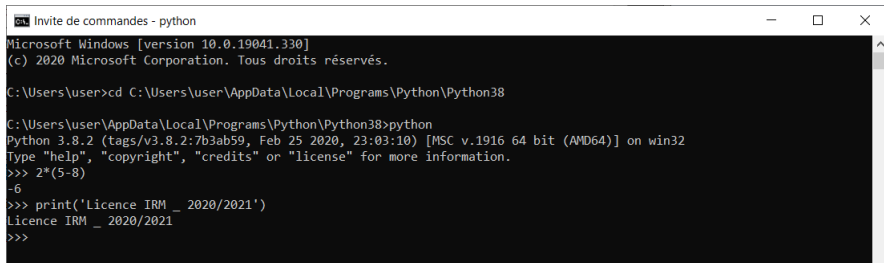
Want to help test development versions of Python 3.15? [Pre-releases](#),  
[Docker images](#)





♦ L'interpréteur **Python** peut être utilisé de deux manières :

✧ En **mode interactif** : on dialogue directement au clavier ; l'interpréteur exécute une ligne puis attend la suivante.



```

C:\Users\user>cd C:\Users\user\AppData\Local\Programs\Python\Python38

C:\Users\user\AppData\Local\Programs\Python\Python38>python
Python 3.8.2 (tags/v3.8.2:7b3ab59, Feb 25 2020, 23:03:10) [MSC v.1916 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> 2*(5-8)
-6
>>> print('Licence IRM _ 2020/2021')
Licence IRM _ 2020/2021
>>>
  
```

✧ Le mode interactif est idéal pour apprendre par expérimentation.

♦ L'interpréteur **Python** peut être utilisé de deux manières :

✧ En **mode script** : l'interpréteur lit un fichier **.py** contenant les instructions et les exécute l'une après l'autre.

```
def pairs(L):
    """Renvoie la liste des nombres pairs de L."""
    return [x for x in L if x % 2 == 0]
```

```
L = [7, 4, 10, 3, 6, 5]
res = pairs(L)
print("Pairs :", res)
print("Combien :", len(res))
```

```
Pairs : [4, 10, 6]
Combien : 3
```

✧ Le mode script permet de conserver et retravailler ses programmes.



# Sommaire

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Avant-propos

Environnement  
Python

Installer Python

**Environnement et  
reproductibilité**

Notion de variable

Types standards

Chaînes de caractères

Affichage

Outils de  
contrôle de flux

Les collections  
de données

Les fonctions

Les modules

Fichiers et  
entrées/sorties

26

De C à Python : une transition

Avant-propos

**Environnement Python**

Installer Python

Environnement et reproductibilité

Notion de variable

Types standards

Chaînes de caractères

Affichage

Outils de contrôle de flux

Les collections de données

Les fonctions

Les modules

Fichiers et entrées/sorties

109

# Éditeurs et IDE : un paysage resserré

- ♦ Pour un développeur venant du C, l'éditeur importe moins que la maîtrise de l'**environnement d'exécution**. On se limite donc à l'essentiel.

| Outil           | Usage                                        | Remarque                                 |
|-----------------|----------------------------------------------|------------------------------------------|
| <b>VS Code</b>  | éditeur polyvalent + extension <b>Python</b> | recommandé aujourd'hui ; le plus utilisé |
| <b>PyCharm</b>  | IDE complet (débogage, refactoring)          | riche mais lourd ; gros projets          |
| <b>Spyder</b>   | IDE scientifique (proche de MATLAB)          | fourni avec Anaconda                     |
| <b>Jupyter</b>  | notebooks : exploration et TP                | mêle code, texte et résultats            |
| <b>Anaconda</b> | distribution (ce n'est pas un IDE)           | fournit conda + paquets scientifiques    |

## Conseil

**VS Code** + l'extension **Python** comme choix par défaut ; **Jupyter** pour l'exploration et les comptes rendus de TP.

Apprendre à coder avec Python

Prof. Abdellah Adib

De C à Python : une transition

Avant-propos

Environnement Python

Installer Python

Environnement et reproductibilité

Notion de variable

Types standards

Chaînes de caractères  
Affichage

Outils de contrôle de flux

Les collections de données

Les fonctions

Les modules

Fichiers et entrées/sorties

# Le vrai enjeu : l'environnement d'exécution

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Avant-propos

Environnement  
Python

Installer Python

Environnement et  
reproductibilité

Notion de variable

Types standards

Chaînes de caractères

Affichage

Outils de  
contrôle de flux

Les collections  
de données

Les fonctions

Les modules

Fichiers et  
entrées/sorties

28

## Problème

« Ça marche sur ma machine. » Deux projets peuvent exiger des versions *différentes* d'une même bibliothèque : tout installer globalement crée des conflits et casse la reproductibilité.

- ◆ En C, vous livrez un **binaire** autonome. En **Python**, le programme dépend de l'**interpréteur** *et* des **versions exactes** des bibliothèques installées.
- ◆ Deux réflexes deviennent indispensables :
  - ✧ **isoler** chaque projet dans son propre environnement ;
  - ✧ **figer** les dépendances pour pouvoir les réinstaller à l'identique.
- ◆ La reproductibilité est une **bonne pratique** attendue (protocole des TP et standard *data science*).

# Isoler : les environnements virtuels (venv)

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Avant-propos

Environnement  
Python

Installer Python

Environnement et  
reproductibilité

Notion de variable

Types standards

Chaînes de caractères

Affichage

Outils de  
contrôle de flux

Les collections  
de données

Les fonctions

Les modules

Fichiers et  
entrées/sorties

29

- ♦ Un **environnement virtuel** contient un interpréteur et des bibliothèques **propres au projet**, isolés du système. Le module **venv** est dans la bibliothèque standard.

```
# creer un environnement dans le dossier du projet
python -m venv .venv

# activer (Linux / macOS)
source .venv/bin/activate
# activer (Windows PowerShell)
.venv\Scripts\Activate.ps1

# desactiver
deactivate
```

- ♦ Une fois activé, l'invite affiche (.venv) : **un environnement par projet.**

# Installer des paquets : pip et PyPI

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Avant-propos

Environnement  
Python

Installer Python

Environnement et  
reproductibilité

Notion de variable

Types standards

Chaînes de caractères

Affichage

Outils de  
contrôle de flux

Les collections  
de données

Les fonctions

Les modules

Fichiers et  
entrées/sorties

30

♦ pip est le gestionnaire de paquets ; **PyPI** est le dépôt public (l'équivalent d'un « apt » pour **Python**).

```
pip install numpy pandas matplotlib    # installer
pip install "numpy==1.26.4"           # version precise
pip list                               # lister l'installé
pip show numpy                         # details d'un paquet
```

## Attention

Toujours installer **dans l'environnement virtuel activé**, jamais dans le **Python** global du système.

# Figer les dépendances : requirements.txt

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition  
Avant-propos  
Environnement  
Python

Installer Python

Environnement et  
reproductibilité

Notion de variable

Types standards

Chaînes de caractères

Affichage

Outils de  
contrôle de flux

Les collections  
de données

Les fonctions

Les modules

Fichiers et  
entrées/sorties

31

✦ Pour qu'un binôme (ou vous-même, plus tard) reconstitue l'environnement à l'identique :

```
# figer les versions exactes de l'environnement courant
pip freeze > requirements.txt
```

```
# réinstaller à l'identique ailleurs
pip install -r requirements.txt
```

requirements.txt

```
numpy==1.26.4
pandas==2.2.2
matplotlib==3.9.0
```

✦ Ce fichier se **versionne avec Git** : c'est la trace exacte des dépendances (traçabilité du protocole).



# Alternative : Conda (Anaconda)

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Avant-propos

Environnement  
Python

Installer Python

Environnement et  
reproductibilité

Notion de variable

Types standards

Chaînes de caractères  
Affichage

Outils de  
contrôle de flux

Les collections  
de données

Les fonctions

Les modules

Fichiers et  
entrées/sorties

32

♦ conda gère à la fois l'environnement et les paquets (y compris non-Python) : pratique en science des données.

```
conda create -n m351 python=3.12      # créer l'environnement
conda activate m351                   # activer
conda install numpy pandas             # installer des paquets
conda env export > environment.yml     # figer
conda env create -f environment.yml     # reproduire ailleurs
```

✧ Choisir **venv + pip** ou **conda** pour un projet donné, sans mélanger les deux.

109

# Reproductibilité : la check-list

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Avant-propos

Environnement  
Python

Installer Python

Environnement et  
reproductibilité

Notion de variable

Types standards

Chaînes de caractères

Affichage

Outils de  
contrôle de flux

Les collections  
de données

Les fonctions

Les modules

Fichiers et  
entrées/sorties

33

♦ Avant de rendre un TP ou de partager un projet, vérifier :

- ♦ **un environnement par projet** (venv ou conda), jamais le **Python** global ;
- ♦ **dépendances figées** (`requirements.txt` / `environment.yml`) et versionnées avec Git ;
- ♦ **graine aléatoire fixée** (ex. `np.random.seed(0)`) pour des résultats reproductibles ;
- ♦ **notebooks** exécutés dans l'ordre, sorties conservées ;
- ♦ un **README** précisant la version de **Python** et la commande d'installation.

## À retenir

Isoler → installer → figer → versionner : un projet reproductible se réinstalle d'une seule commande.



# Sommaire

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Avant-propos

Environnement  
Python

Installer Python

Environnement et  
reproductibilité

Notion de variable

Types standards

Chaînes de caractères

Affichage

Outils de  
contrôle de flux

Les collections  
de données

Les fonctions

Les modules

Fichiers et  
entrées/sorties

34

109

De C à Python : une transition

Avant-propos

**Environnement Python**

Installer Python

Environnement et reproductibilité

Notion de variable

Types standards

Chaînes de caractères

Affichage

Outils de contrôle de flux

Les collections de données

Les fonctions

Les modules

Fichiers et entrées/sorties

# Identificateurs et variables

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Avant-propos

Environnement  
Python

Installer Python

Environnement et  
reproductibilité

Notion de variable

Types standards

Chaînes de caractères  
Affichage

Outils de  
contrôle de flux

Les collections  
de données

Les fonctions

Les modules

Fichiers et  
entrées/sorties

35

- ♦ Une **variable** désigne un emplacement mémoire contenant une information (nombre, texte, liste, ...).
- ♦ On crée une variable avec le signe **=**, suivi de la valeur ; **print()** affiche son contenu.

```
>>> formation = "Licence IRM"  
>>> print(formation)  
Licence IRM
```

- ♦ Un nom commence par une lettre ou **\_**, puis lettres, chiffres ou **\_**.
- ♦ **Python** distingue les majuscules des minuscules : **LIC\_IRM**  $\neq$  **lic\_irm**.

109

# Mots-clés réservés (Python 3)

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Avant-propos

Environnement  
Python

Installer Python

Environnement et  
reproductibilité

Notion de variable

Types standards

Chaînes de caractères

Affichage

Outils de  
contrôle de flux

Les collections  
de données

Les fonctions

Les modules

Fichiers et  
entrées/sorties

36

- ◆ Les identificateurs suivants sont réservés et ne peuvent pas servir de noms de variables :

|          |         |       |       |        |
|----------|---------|-------|-------|--------|
| False    | None    | True  | and   | as     |
| assert   | async   | await | break | class  |
| continue | def     | del   | elif  | else   |
| except   | finally | for   | from  | global |
| if       | import  | in    | is    | lambda |
| nonlocal | not     | or    | pass  | raise  |
| return   | try     | while | with  | yield  |

- ✧ `print()` n'est **pas** un mot-clé : c'est une fonction intégrée.

109



# Sommaire

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Avant-propos

Environnement  
Python

Installer Python

Environnement et  
reproductibilité

Notion de variable

**Types standards**

Chaînes de caractères

Affichage

Outils de  
contrôle de flux

Les collections  
de données

Les fonctions

Les modules

Fichiers et  
entrées/sorties

37

109

De C à Python : une transition

Avant-propos

**Environnement Python**

Installer Python

Environnement et reproductibilité

Notion de variable

**Types standards**

Chaînes de caractères

Affichage

Outils de contrôle de flux

Les collections de données

Les fonctions

Les modules

Fichiers et entrées/sorties

# Le type des variables

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition  
Avant-propos  
Environnement  
Python

Installer Python

Environnement et  
reproductibilité

Notion de variable

Types standards

Chaînes de caractères

Affichage

Outils de  
contrôle de flux

Les collections  
de données

Les fonctions

Les modules

Fichiers et  
entrées/sorties

38

109

## Important

Le typage est **dynamique** : le type n'est pas déclaré, il est lié à la valeur manipulée, pas à la variable.

- ♦ C'est au programmeur de suivre le type courant d'une variable pour éviter les opérations incompatibles.
- ♦ La fonction `type()` renvoie le type d'une valeur :

```
>>> formation = "Licence IRM"  
>>> type(formation)  
<class 'str'>
```

# Les types de base

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Avant-propos

Environnement  
Python

Installer Python

Environnement et  
reproductibilité

Notion de variable

Types standards

Chaînes de caractères

Affichage

Outils de  
contrôle de flux

Les collections  
de données

Les fonctions

Les modules

Fichiers et  
entrées/sorties

39

♦ Aperçu des types les plus fréquents en **Python** :

| Type     | Exemple                        | Commentaire                   |
|----------|--------------------------------|-------------------------------|
| bool     | <code>actif = True</code>      | valeurs True / False          |
| int      | <code>rayon = 6_371_000</code> | entiers à précision illimitée |
| float    | <code>pi = 3.14159</code>      | nombres à virgule flottante   |
| str      | <code>licence = "IRM"</code>   | texte (chaîne immuable)       |
| complex  | <code>onde = 3 + 6j</code>     | nombres complexes             |
| NoneType | <code>stock = None</code>      | absence de valeur             |

✧ Le type long du C n'existe pas : un int n'a pas de borne.

109



♦ **Python** fournit les opérateurs arithmétiques usuels, avec une notation courte et une notation longue.

| Opération      | Courte               | Longue                  | Commentaire                           |
|----------------|----------------------|-------------------------|---------------------------------------|
| Addition       | <code>a += 1</code>  | <code>a = a + 1</code>  |                                       |
| Multiplication | <code>a *= 2</code>  | <code>a = a * 2</code>  |                                       |
| Division       | <code>a /= 2</code>  | <code>a = a / 2</code>  | division réelle (résultat float)      |
| Division ent.  | <code>a //= 2</code> | <code>a = a // 2</code> | quotient entier (arrondi vers le bas) |
| Modulo         | <code>a %= 2</code>  | <code>a = a % 2</code>  | reste de la division                  |
| Puissance      | <code>a **= 2</code> | <code>a = a ** 2</code> | équivalent à $a^2$                    |

```
>>> 3 / 2      # division réelle -> 1.5   (en C, 3/2 vaut 1)
>>> 3 // 2     # division entière -> 1
```



# Sommaire

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Avant-propos

Environnement  
Python

Installer Python

Environnement et  
reproductibilité

Notion de variable

Types standards

Chaînes de caractères

Affichage

Outils de  
contrôle de flux

Les collections  
de données

Les fonctions

Les modules

Fichiers et  
entrées/sorties

41

109

De C à Python : une transition

Avant-propos

Environnement Python

Installer Python

Environnement et reproductibilité

Notion de variable

Types standards

Chaînes de caractères

Affichage

Outils de contrôle de flux

Les collections de données

Les fonctions

Les modules

Fichiers et entrées/sorties

- ♦ On accède à chaque caractère par son **indice** : à partir de 0 depuis le début, de -1 depuis la fin.

|                            |     |     |    |    |    |    |    |    |    |    |    |
|----------------------------|-----|-----|----|----|----|----|----|----|----|----|----|
| <code>senti = "</code>     | I   |     | f  | e  | e  | l  |    | g  | o  | o  | d  |
| <code>indice ≥ 0</code>    | 0   | 1   | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 |
| <code>indice &lt; 0</code> | -11 | -10 | -9 | -8 | -7 | -6 | -5 | -4 | -3 | -2 | -1 |

```
>>> senti = "I feel good"
>>> senti[2]      # 'f'
>>> senti[-4]     # 'g'
>>> senti[-11]    # 'I'
```

# Découpage en tranches (slicing)

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Avant-propos

Environnement  
Python

Installer Python

Environnement et  
reproductibilité

Notion de variable

Types standards

Chaînes de caractères

Affichage

Outils de  
contrôle de flux

Les collections  
de données

Les fonctions

Les modules

Fichiers et  
entrées/sorties

43

109

♦ On extrait une sous-chaîne par `[début:fin]` ; la borne `fin` est **exclue**.

```
>>> formation = "Licence IRM"
>>> formation[2:7]      # indices 2 a 6 (7 exclu)
'cence'
```

♦ L'opérateur `in` teste la présence d'une sous-chaîne :

```
>>> "cence" in formation      # True
>>> "L" in formation[1:3]    # False
```

# Concaténation et méthodes usuelles

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition  
Avant-propos  
Environnement  
Python

Installer Python

Environnement et  
reproductibilité

Notion de variable

Types standards

Chaînes de caractères

Affichage

Outils de  
contrôle de flux

Les collections  
de données

Les fonctions

Les modules

Fichiers et  
entrées/sorties

44

```
>>> formation = "New Licence IRM"
>>> promo = "First promotion"
>>> actuel = formation + ", " + promo
>>> print(actuel)
New Licence IRM, First promotion
```

```
>>> len("anticonstitutionnellement") # 25
>>> "Recto-Verso".lower()           # 'recto-verso'
>>> "licence".upper()                # 'LICENCE'
>>> "python".capitalize()           # 'Python'
>>> "  Bienvenue !  ".strip()        # 'Bienvenue !'
>>> "maman aime manger".count("ma") # 3
```

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition  
Avant-propos  
Environnement  
Python

Installer Python

Environnement et  
reproductibilité

Notion de variable

Types standards

Chaînes de caractères

Affichage

Outils de  
contrôle de flux

Les collections  
de données

Les fonctions

Les modules

Fichiers et  
entrées/sorties

♦ Un aperçu des méthodes les plus utilisées :

| Méthode                                                | Description                                               |
|--------------------------------------------------------|-----------------------------------------------------------|
| <code>count()</code>                                   | compte les occurrences d'un motif                         |
| <code>find()</code>                                    | renvoie la position de la première occurrence             |
| <code>replace()</code>                                 | remplace un motif par un autre                            |
| <code>split()</code>                                   | découpe la chaîne selon un séparateur (renvoie une liste) |
| <code>join()</code>                                    | assemble un itérable en une chaîne                        |
| <code>startswith()</code> /<br><code>endswith()</code> | teste le début / la fin (renvoie un booléen)              |
| <code>lower()</code> /<br><code>upper()</code>         | minuscules / majuscules                                   |
| <code>strip()</code>                                   | supprime les espaces en début et fin                      |



# Sommaire

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Avant-propos

Environnement  
Python

Installer Python

Environnement et  
reproductibilité

Notion de variable

Types standards

Chaînes de caractères

Affichage

Outils de  
contrôle de flux

Les collections  
de données

Les fonctions

Les modules

Fichiers et  
entrées/sorties

De C à Python : une transition

Avant-propos

**Environnement Python**

Installer Python

Environnement et reproductibilité

Notion de variable

Types standards

Chaînes de caractères

Affichage

Outils de contrôle de flux

Les collections de données

Les fonctions

Les modules

Fichiers et entrées/sorties

46

109

# Écriture formatée : les f-strings (1/3)

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Avant-propos

Environnement  
Python

Installer Python

Environnement et  
reproductibilité

Notion de variable

Types standards

Chaînes de caractères

Affichage

47

Outils de  
contrôle de flux

Les collections  
de données

Les fonctions

Les modules

Fichiers et  
entrées/sorties

109

- ♦ Une **f-string** est une chaîne préfixée par **f** : ce qui est entre **{ }** est **évalué** puis inséré dans le texte.

```
>>> nom, age = "Ahmed", 32
>>> print(f"{nom} a {age} ans")
Ahmed a 32 ans
```

*# entre accolades on peut mettre une EXPRESSION, pas juste une variable*

```
>>> print(f"L'an prochain : {age + 1} ans")
L'an prochain : 33 ans
```

```
>>> prix, quantite = 4.5, 3
>>> print(f"Total = {prix * quantite} DH")
Total = 13.5 DH
```

- ♦ Sans le **f**, rien n'est remplacé : "age" affiche littéralement {age}.



# Écriture formatée : les f-strings (2/3)

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition  
Avant-propos  
Environnement  
Python

Installer Python  
Environnement et  
reproductibilité  
Notion de variable  
Types standards  
Chaînes de caractères  
Affichage

48

Outils de  
contrôle de flux  
Les collections  
de données  
Les fonctions  
Les modules  
Fichiers et  
entrées/sorties

109

♦ Après un `:` on précise le **format** (comme le `%` de `printf` en C).

```
>>> pi = 3.14159265
>>> print(f"{pi:.2f}")    # 2 decimales
3.14
>>> print(f"{1000000:},")
1,000,000
>>> print(f"{0.25:.1%}")
25.0%
>>> print(f"{42:05d}")
00042
```

| Format            | Effet                     | Format            | Effet                  |
|-------------------|---------------------------|-------------------|------------------------|
| <code>:.2f</code> | flottant, 2 décimales     | <code>: ,</code>  | séparateur de milliers |
| <code>:d</code>   | entier                    | <code>:.1%</code> | pourcentage            |
| <code>:05d</code> | largeur 5, zéros à gauche |                   |                        |

# Écriture formatée : les f-strings (3/3)

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Avant-propos

Environnement  
Python

Installer Python

Environnement et  
reproductibilité

Notion de variable

Types standards

Chaînes de caractères

Affichage

49

Outils de  
contrôle de flux

Les collections  
de données

Les fonctions

Les modules

Fichiers et  
entrées/sorties

109

- ◆ **Alignement** sur une largeur donnée (< gauche, > droite, ^ centré) — utile pour des tableaux :

```
>>> print(f"{'Nom':<10}|{'Age':>5}")
Nom      |   Age
>>> print(f"{'Ahmed':<10}|{'32':>5}")
Ahmed    |   32
```

- ◆ Astuce de **débogage** (Python 3.8+) : {x=} affiche le nom *et* la valeur.

```
>>> x = 5
>>> print(f"{x=}")
x=5
>>> print(f"{x*2=}")
x*2=10
```

- ◆ Les anciennes écritures "...".format(...) et "%d" % x restent valides mais sont moins lisibles.

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Avant-propos

Environnement  
Python

Installer Python

Environnement et  
reproductibilité

Notion de variable

Types standards

Chaînes de caractères

Affichage

50

Outils de  
contrôle de flux

Les collections  
de données

Les fonctions

Les modules

Fichiers et  
entrées/sorties

109

◆ Les commentaires aident à la lecture et sont ignorés par l'interpréteur.

✧ Sur une seule ligne : caractère # en début de zone.

✧ Sur plusieurs lignes : triple guillemets """ ...""".

```
# Commentaire sur une seule ligne
x = 42           # commentaire en fin de ligne
```

```
"""
Commentaire
sur plusieurs lignes
"""
```



# Sommaire

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Avant-propos

Environnement  
Python

Outils de  
contrôle de flux

Le contrôle de  
conditions

Les boucles

Les collections  
de données

Les fonctions

Les modules

Fichiers et  
entrées/sorties

De C à Python : une transition

Avant-propos

Environnement Python

**Outils de contrôle de flux**

Le contrôle de conditions

Les boucles

Les collections de données

Les fonctions

Les modules

Fichiers et entrées/sorties

51

109

# Les branchements conditionnels

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Avant-propos

Environnement  
Python

Outils de  
contrôle de flux

Le contrôle de  
conditions

Les boucles

Les collections  
de données

Les fonctions

Les modules

Fichiers et  
entrées/sorties

52

- ◆ Un programme peut suivre différentes directions selon le résultat d'un test booléen (**Vrai** / **Faux**).
- ◆ Le test est suivi de **:** et les instructions dépendantes sont **indentées** (décalées vers la droite).
- ◆ Trois formes de sélection :
  - ✧ à sens unique : **if**;
  - ✧ à deux sens : **if ... else**;
  - ✧ multiple : **if ... elif ... else**.

109

# Sélection : if / else / elif

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Avant-propos

Environnement  
Python

Outils de  
contrôle de flux

Le contrôle de  
conditions

Les boucles

Les collections  
de données

Les fonctions

Les modules

Fichiers et  
entrées/sorties

53

109

## Sens unique

```
if condition:  
    bloc_si
```

## Deux sens

```
if condition:  
    bloc_si  
else:  
    bloc_sinon
```

## Multiple

```
if cond1:  
    bloc1  
elif cond2:  
    bloc2  
else:  
    bloc3
```

♦ **elif** enchaîne plusieurs tests successifs (alternative multiple).



# Sommaire

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Avant-propos

Environnement  
Python

Outils de  
contrôle de flux

Le contrôle de  
conditions

Les boucles

Les collections  
de données

Les fonctions

Les modules

Fichiers et  
entrées/sorties

De C à Python : une transition

Avant-propos

Environnement Python

**Outils de contrôle de flux**

Le contrôle de conditions

Les boucles

Les collections de données

Les fonctions

Les modules

Fichiers et entrées/sorties

54

109

# Traitement répétitif : while et for

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Avant-propos

Environnement  
Python

Outils de  
contrôle de flux

Le contrôle de  
conditions

Les boucles

Les collections  
de données

Les fonctions

Les modules

Fichiers et  
entrées/sorties

55

♦ **while** : itération **indéfinie** (répète tant qu'une condition est vraie).

♦ **for** : itération **définie** (parcourt un itérable : range, liste, chaîne, ...).

## Boucle while

```
n = 5
while n > 0:
    print(n)
    n -= 1
```

## Boucle for

```
for i in range(1, 6):
    print(i)
```

♦ **range(a, b)** parcourt a, a+1, ..., b-1 (la borne b est exclue).



# Clause else, break et continue

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Avant-propos

Environnement  
Python

Outils de  
contrôle de flux

Le contrôle de  
conditions

Les boucles

Les collections  
de données

Les fonctions

Les modules

Fichiers et  
entrées/sorties

- ♦ Une clause **else** peut suivre une boucle : elle s'exécute si la boucle se termine **sans break**.
- ♦ **break** interrompt la boucle ; **continue** passe à l'itération suivante.

56

```
for x in range(10):  
    if x == 5:  
        break           # sort de la boucle  
    print(x)
```

```
for x in range(6):  
    if x % 2 == 0:  
        continue       # ignore les pairs  
    print(x)
```

109



# Sommaire

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Avant-propos

Environnement  
Python

Outils de  
contrôle de flux

Les collections  
de données

Généralités

Les listes

Les tuples

Les ensembles

Les dictionnaires

Les fonctions

Les modules

Fichiers et  
entrées/sorties

De C à Python : une transition

Avant-propos

Environnement Python

Outils de contrôle de flux

**Les collections de données**

Généralités

Les listes

Les tuples

Les ensembles

Les dictionnaires

Les fonctions

Les modules

Fichiers et entrées/sorties

57

109

# Types de collections

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Avant-propos  
Environnement  
Python

Outils de  
contrôle de flux

Les collections  
de données

Généralités

Les listes

Les tuples

Les ensembles

Les dictionnaires

Les fonctions

Les modules

Fichiers et  
entrées/sorties

58

109

- ◆ **Python** offre quatre types de données composées (séquences) :
  - ✧ **List** : ordonnée et modifiable ; doublons autorisés.
  - ✧ **Tuple** : ordonné et non modifiable (immuable) ; doublons autorisés.
  - ✧ **Set** : non ordonné, non indexé ; pas de doublons.
  - ✧ **Dictionary** : paires **clé :valeur**, modifiable ; clés uniques.
- ◆ Choisir le bon type améliore l'efficacité du traitement et la clarté du code.



# Sommaire

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Avant-propos

Environnement  
Python

Outils de  
contrôle de flux

Les collections  
de données

Généralités

**Les listes**

Les tuples

Les ensembles

Les dictionnaires

Les fonctions

Les modules

Fichiers et  
entrées/sorties

De C à Python : une transition

Avant-propos

Environnement Python

Outils de contrôle de flux

**Les collections de données**

Généralités

Les listes

Les tuples

Les ensembles

Les dictionnaires

Les fonctions

Les modules

Fichiers et entrées/sorties

59

109

# Listes : création et accès

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Avant-propos

Environnement  
Python

Outils de  
contrôle de flux

Les collections  
de données

Généralités

Les listes

Les tuples

Les ensembles

Les dictionnaires

Les fonctions

Les modules

Fichiers et  
entrées/sorties

60

- ◆ Une liste contient une série d'éléments (éventuellement hétérogènes), entre crochets, séparés par des virgules.
- ◆ L'accès se fait par indice (à partir de 0) ; le découpage fonctionne comme pour les chaînes.

```
>>> liste1 = ["Python", "langage", 1996, 2020]
>>> liste1[0]          # 'Python'
>>> carre = [1, 4, 9, 16, 25, 36, 49, 64]
>>> carre[3:8]         # [16, 25, 36, 49, 64]
>>> carre[2:]          # [9, 16, 25, 36, 49, 64]
>>> carre[:5]          # [1, 4, 9, 16, 25]
```

109

# Listes : ajout et suppression

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Avant-propos

Environnement  
Python

Outils de  
contrôle de flux

Les collections  
de données

Généralités

Les listes

Les tuples

Les ensembles

Les dictionnaires

Les fonctions

Les modules

Fichiers et  
entrées/sorties

61

```
>>> ages = [54, 89, 14, 25, 64, 71]
>>> ages.append(30)           # ajoute a la fin
>>> ages.insert(2, 100)       # insere a la position 2
>>> print(ages)
[54, 89, 100, 14, 25, 64, 71, 30]
```

```
>>> presents = ["Ali", "Mohammed", "Anas", "Wiam", "Yosra"]
>>> del presents[0]           # supprime par indice
>>> presents.remove("Yosra")  # supprime par valeur (renvoie None)
>>> print(presents)
['Mohammed', 'Anas', 'Wiam']
```

♦ `remove()` modifie la liste en place et renvoie `None` : ne pas afficher son résultat.

# Parcourir une liste : par indice ou par contenu

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Avant-propos

Environnement  
Python

Outils de  
contrôle de flux

Les collections  
de données

Généralités

Les listes

Les tuples

Les ensembles

Les dictionnaires

Les fonctions

Les modules

Fichiers et  
entrées/sorties

- ♦ En C, on parcourt **toujours par indice**. Python permet en plus le parcours **par contenu** (*for-each*) : on obtient directement les éléments.

## Par indice (réflexe C)

```
fruits = ["pomme", "kiwi", "fig"]
for i in range(len(fruits)):
    print(i, fruits[i])
# 0 pomme / # 1 kiwi / # 2 fig
```

## Par contenu (idiome Python)

```
for fruit in fruits:
    print(fruit)
# pomme / # kiwi / # fig
```

- ♦ Besoin de l'indice ET de la valeur? `enumerate()` :

```
for i, fruit in enumerate(fruits):
    print(i, fruit)
# 0 pomme / 1 kiwi / 2 fig
```

## Bon réflexe

Préférez le parcours **par contenu** ; utilisez `enumerate()` quand l'indice est nécessaire. Le `for i in range(len(...))` « à la C » est rarement utile en Python.

# Méthodes des listes (1/2) — ajout et suppression

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Avant-propos

Environnement  
Python

Outils de  
contrôle de flux

Les collections  
de données

Généralités

**Les listes**

Les tuples

Les ensembles

Les dictionnaires

Les fonctions

Les modules

Fichiers et  
entrées/sorties

♦ Une **méthode** s'appelle sur l'objet avec un point : `liste.methode(...)`.

| Méthode                  | Rôle                                            | Exemple → résultat                       |
|--------------------------|-------------------------------------------------|------------------------------------------|
| <code>append(x)</code>   | ajoute <code>x</code> à la fin                  | <code>[1,2].append(3) → [1,2,3]</code>   |
| <code>insert(i,x)</code> | insère <code>x</code> à l'indice <code>i</code> | <code>[1,3].insert(1,2) → [1,2,3]</code> |
| <code>extend(it)</code>  | ajoute chaque élément d'un itérable             | <code>[1].extend([2,3]) → [1,2,3]</code> |
| <code>remove(x)</code>   | retire la 1re occurrence de <code>x</code>      | <code>[1,2,2].remove(2) → [1,2]</code>   |
| <code>pop([i])</code>    | retire <b>et renvoie</b> (défaut : dernier)     | <code>[1,2,3].pop() → 3</code>           |
| <code>clear()</code>     | vide la liste                                   | <code>[1,2].clear() → []</code>          |

♦ Ces méthodes (sauf `pop`) modifient la liste **en place** et renvoient `None`.



# Méthodes des listes (2/2) — recherche, tri, copie

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition  
Avant-propos  
Environnement  
Python  
Outils de  
contrôle de flux  
Les collections  
de données

Généralités

**Les listes**

Les tuples

Les ensembles

Les dictionnaires

Les fonctions

Les modules

Fichiers et  
entrées/sorties

64

| Méthode                | Rôle                               | Exemple → résultat                       |
|------------------------|------------------------------------|------------------------------------------|
| <code>index(x)</code>  | indice de la 1re occurrence        | <code>[7,8,9].index(8) → 1</code>        |
| <code>count(x)</code>  | nombre d'occurrences               | <code>[1,2,2].count(2) → 2</code>        |
| <code>sort()</code>    | trie <b>en place</b>               | <code>[3,1,2].sort() → [1,2,3]</code>    |
| <code>reverse()</code> | inverse <b>en place</b>            | <code>[1,2,3].reverse() → [3,2,1]</code> |
| <code>copy()</code>    | copie superficielle (nouvel objet) | <code>[1,2].copy() → [1,2]</code>        |

♦ `sort` accepte `key=` (critère) et `reverse=True` : ex. `L.sort(key=len, reverse=True)`.

♦ `index(x)` lève `ValueError` si `x` est absent (tester avec `in` d'abord).

# Fonctions intégrées sur les listes (1/2)

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Avant-propos

Environnement  
Python

Outils de  
contrôle de flux

Les collections  
de données

Généralités

Les listes

Les tuples

Les ensembles

Les dictionnaires

Les fonctions

Les modules

Fichiers et  
entrées/sorties

♦ Une fonction intégrée prend la liste en **argument** : `fonction(liste)`.

| Fonction                   | Rôle                        | Exemple → résultat                         |
|----------------------------|-----------------------------|--------------------------------------------|
| <code>len(L)</code>        | nombre d'éléments           | <code>len([1,2,3]) → 3</code>              |
| <code>sum(L)</code>        | somme des éléments          | <code>sum([1,2,3]) → 6</code>              |
| <code>min(L)/max(L)</code> | plus petit / plus grand     | <code>max([4,9,2]) → 9</code>              |
| <code>sorted(L)</code>     | <b>nouvelle</b> liste triée | <code>sorted([3,1,2]) → [1,2,3]</code>     |
| <code>reversed(L)</code>   | itérateur inversé           | <code>list(reversed([1,2])) → [2,1]</code> |
| <code>list(it)</code>      | construit une liste         | <code>list("abc") → ['a','b','c']</code>   |

♦ `sorted/reversed` renvoient un **nouvel** objet; `sort/reverse` (méthodes) modifient en place.

♦ Fonctions renvoyant un **itérateur** : on les enveloppe souvent dans `list(...)` pour voir le résultat.

```
>>> list(enumerate(["a", "b"]))           # (indice, valeur)
[(0, 'a'), (1, 'b')]
>>> list(zip([1, 2], ["a", "b"]))         # apparie deux listes
[(1, 'a'), (2, 'b')]
>>> list(map(str, [1, 2, 3]))             # applique une fonction
['1', '2', '3']
>>> list(filter(lambda x: x > 1, [1, 2, 3])) # garde si condition
[2, 3]
>>> any([0, 0, 3]), all([1, 2, 0])        # au moins un / tous vrais
(True, False)
```

## À noter

`map/filter` ont souvent un équivalent en **compréhension**, jugé plus lisible : `[str(x) for x in L]`, `[x for x in L if x > 1]`.



# Sommaire

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Avant-propos

Environnement  
Python

Outils de  
contrôle de flux

Les collections  
de données

Généralités

Les listes

**Les tuples**

Les ensembles

Les dictionnaires

Les fonctions

Les modules

Fichiers et  
entrées/sorties

De C à Python : une transition

Avant-propos

Environnement Python

Outils de contrôle de flux

**Les collections de données**

Généralités

Les listes

**Les tuples**

Les ensembles

Les dictionnaires

Les fonctions

Les modules

Fichiers et entrées/sorties

67

109

# Tuples : structure immuable

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Avant-propos

Environnement  
Python

Outils de  
contrôle de flux

Les collections  
de données

Généralités

Les listes

Les tuples

Les ensembles

Les dictionnaires

Les fonctions

Les modules

Fichiers et  
entrées/sorties

68

- ♦ Un tuple ressemble à une liste, mais on ne peut **ni modifier, ni ajouter, ni retirer** ses éléments.
- ♦ Compact et rapide d'accès ; parenthèses optionnelles ; tuple d'un seul élément : (a,).

```
>>> tupl = "Arabe", "Francais", "Anglais", "Allemand"
>>> tupl[1:2]                                # ('Francais',)
>>> tupl[1] = "Chinois"
TypeError: 'tuple' object does not support item assignment
>>> del tupl                                # on peut supprimer le tuple entier
```

# Les tuples : à quoi servent-ils ?

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Avant-propos

Environnement  
Python

Outils de  
contrôle de flux

Les collections  
de données

Généralités

Les listes

**Les tuples**

Les ensembles

Les dictionnaires

Les fonctions

Les modules

Fichiers et  
entrées/sorties

69

- ◆ Un tuple est une séquence **immuable** : cette contrainte est en réalité un **atout**.
  - ✧ **Sûr** : des données qui ne doivent pas changer (coordonnées, couleur RVB, constantes) sont protégées contre toute modification accidentelle.
  - ✧ **Hachable** : un tuple peut servir de **clé de dictionnaire** ou d'**élément d'ensemble** — une liste ne le peut pas.
  - ✧ **Léger** : plus compact et légèrement plus rapide qu'une liste.
- ◆ Usages phares : **retour multiple**, **unpacking**/échange, **clés composées**, **enregistrements**, résultats de `zip()`/`enumerate()`.

## Règle simple

**Liste** = collection homogène que l'on *modifie*.    **Tuple** = enregistrement *figé*, souvent hétérogène.

109

# Utilité (1) : retour multiple et unpacking

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Avant-propos

Environnement  
Python

Outils de  
contrôle de flux

Les collections  
de données

Généralités

Les listes

**Les tuples**

Les ensembles

Les dictionnaires

Les fonctions

Les modules

Fichiers et  
entrées/sorties

♦ Une fonction « renvoie plusieurs valeurs » : en réalité, elle renvoie **un tuple**, qu'on récupère par **unpacking**.

```
>>> def minmax(valeurs):
...     return min(valeurs), max(valeurs)    # renvoie un TUPLE
>>> bas, haut = minmax([4, 9, 2, 7])        # unpacking
>>> bas, haut
(2, 9)

>>> x, y = 1, 2
>>> x, y = y, x                            # échange SANS variable temporaire
>>> (x, y)
(2, 1)

>>> premier, *reste = [10, 20, 30, 40]    # unpacking étendu
>>> premier, reste
(10, [20, 30, 40])
```

# Utilité (2) : clés composées (ce que la liste ne peut pas)

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Avant-propos

Environnement  
Python

Outils de  
contrôle de flux

Les collections  
de données

Généralités

Les listes

Les tuples

Les ensembles

Les dictionnaires

Les fonctions

Les modules

Fichiers et  
entrées/sorties

- ♦ Étant immuable, un tuple est **hachable** : idéal comme **clé** d'un dictionnaire indexé par coordonnées.

```
>>> # une grille creuse : (ligne, colonne) -> valeur
>>> grille = {(0, 0): "A", (1, 2): "B", (2, 1): "C"}
>>> grille[(1, 2)]
'B'
>>> (0, 0) in grille
True
```

```
>>> # une liste (mutable) ne peut PAS etre une cle :
>>> {[0, 0]: "A"}
TypeError: unhashable type: 'list'
```

- ♦ Autres usages : arêtes d'un graphe (u, v), mémoïsation, comptage de paires.



# Utilité (3) : enregistrements structurés

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Avant-propos

Environnement  
Python

Outils de  
contrôle de flux

Les collections  
de données

Généralités

Les listes

Les tuples

Les ensembles

Les dictionnaires

Les fonctions

Les modules

Fichiers et  
entrées/sorties

72

```
>>> etudiant = ("Sara", 20, "Rabat")    # un enregistrement heterogene
>>> nom, age, ville = etudiant          # unpacking lisible
>>> print(nom, "-", age, "ans")
Sara - 20 ans

>>> # zip et enumerate produisent des tuples
>>> for nom, note in zip(["Ali", "Sara"], [15, 18]):
...     print(nom, note)
Ali 15
Sara 18
```

## Pour aller plus loin : namedtuple

```
from collections import namedtuple
Point = namedtuple("Point", ["x", "y"]); p = Point(3, 5)
p.x vaut 3 : un enregistrement immuable dont les champs ont un nom (plus lisible que p[0]).
```



# Sommaire

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Avant-propos

Environnement  
Python

Outils de  
contrôle de flux

Les collections  
de données

Généralités

Les listes

Les tuples

**Les ensembles**

Les dictionnaires

Les fonctions

Les modules

Fichiers et  
entrées/sorties

De C à Python : une transition

Avant-propos

Environnement Python

Outils de contrôle de flux

**Les collections de données**

Généralités

Les listes

Les tuples

**Les ensembles**

Les dictionnaires

Les fonctions

Les modules

Fichiers et entrées/sorties

73

109

# Ensembles (set)

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Avant-propos

Environnement  
Python

Outils de  
contrôle de flux

Les collections  
de données

Généralités

Les listes

Les tuples

Les ensembles

Les dictionnaires

Les fonctions

Les modules

Fichiers et  
entrées/sorties

74

◆ Un **set** est une collection non ordonnée, non indexée, **sans doublon**, notée entre accolades.

◆ Idéal pour la déduplication et les tests d'appartenance.

```
>>> couleurs = {"rouge", "vert", "bleu"}
>>> couleurs.add("rouge")      # doublon ignore
>>> "vert" in couleurs         # True
>>> set([1, 2, 2, 3, 3, 3])    # {1, 2, 3}
```

109

# Les ensembles : à quoi servent-ils ?

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Avant-propos  
Environnement  
Python

Outils de  
contrôle de flux

Les collections  
de données

Généralités

Les listes

Les tuples

Les ensembles

Les dictionnaires

Les fonctions

Les modules

Fichiers et  
entrées/sorties

75

- ◆ Un set est une collection d'éléments **uniques**, **non ordonnée** et **non indexée**.
  - ✧ **Unicité automatique** : les doublons disparaissent (dédoublonnage).
  - ✧ **Appartenance très rapide** :  $x \in s$  est quasi instantané, même sur un grand ensemble (grâce au hachage).
  - ✧ **Opérations d'ensembles** : union, intersection, différence — comme en mathématiques.
- ◆ En contrepartie : pas d'ordre, pas d'indice ( $s[0]$  interdit) ; les éléments doivent être **hachables**.

## Règle simple

**Liste** = séquence ordonnée, doublons permis.    **Set** = éléments *uniques*, pour *tester l'appartenance* et *combiner* des ensembles.

109

# Utilité (1) : dédoublonnage et appartenance rapide

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Avant-propos

Environnement  
Python

Outils de  
contrôle de flux

Les collections  
de données

Généralités

Les listes

Les tuples

Les ensembles

Les dictionnaires

Les fonctions

Les modules

Fichiers et  
entrées/sorties

76

```
>>> # dedoublonnage instantane d'une liste
>>> notes = [12, 15, 12, 9, 15, 18]
>>> set(notes)                # l'ordre d'affichage n'est pas garanti
{9, 12, 15, 18}
>>> len(set(notes))           # nombre de valeurs DISTINCTES
4

>>> # test d'appartenance : rapide, meme sur un grand ensemble
>>> inscrits = {"CNE100", "CNE205", "CNE310"}
>>> "CNE205" in inscrits
True
>>> "CNE999" in inscrits
False
```

♦ `x in s` est quasi  $O(1)$  sur un set (hachage), contre  $O(n)$  sur une liste : décisif sur de grandes collections.

## Utilité (2) : combiner des ensembles

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Avant-propos

Environnement  
Python

Outils de  
contrôle de flux

Les collections  
de données

Généralités

Les listes

Les tuples

Les ensembles

Les dictionnaires

Les fonctions

Les modules

Fichiers et  
entrées/sorties

- ◆ Deux groupes d'étudiants (maths, physique) : les opérations d'ensembles répondent directement aux questions courantes.

```
>>> maths = {"Ali", "Sara", "Omar", "Wiam"}
>>> physique = {"Sara", "Omar", "Nabil"}
>>> maths | physique      # UNION : tous les etudiants
{'Ali', 'Sara', 'Omar', 'Wiam', 'Nabil'}
>>> maths & physique      # INTERSECTION : inscrits aux deux
{'Sara', 'Omar'}
>>> maths - physique      # DIFFERENCE : maths seulement
{'Ali', 'Wiam'}
>>> maths ^ physique      # une seule des deux matieres
{'Ali', 'Wiam', 'Nabil'}
```

## Utilité (3) : comparaisons et vocabulaire unique

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Avant-propos

Environnement  
Python

Outils de  
contrôle de flux

Les collections  
de données

Généralités

Les listes

Les tuples

Les ensembles

Les dictionnaires

Les fonctions

Les modules

Fichiers et  
entrées/sorties

78

```
>>> acquis = {"algo", "C", "maths"}
>>> requis = {"algo", "C"}
>>> requis <= acquis           # requis est-il inclus dans acquis ?
True
>>> requis.issubset(acquis)    # forme lisible equivalente
True
>>> acquis.isdisjoint({"reseau"}) # aucun element commun ?
True

>>> # mots DISTINCTS d'un texte, en une ligne
>>> texte = "le chat le chien le chat"
>>> set(texte.split())
{'le', 'chat', 'chien'}
```

### Cas d'usage typiques

Vérifier des **prérequis** (inclusion), détecter des **doublons**, extraire un **vocabulaire**, calculer des **éléments communs** entre deux jeux de données.

109

# Set ou list : quand choisir ?

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Avant-propos  
Environnement  
Python

Outils de  
contrôle de flux

Les collections  
de données

Généralités

Les listes

Les tuples

Les ensembles

Les dictionnaires

Les fonctions

Les modules

Fichiers et  
entrées/sorties

79

| Critère                | list               | set                        |
|------------------------|--------------------|----------------------------|
| Ordre                  | ordonnée (indices) | non ordonné                |
| Accès par indice       | oui (L[0])         | non (s[0] interdit)        |
| Doublons               | autorisés          | interdits (uniques)        |
| Modifiable             | oui                | oui                        |
| Appartenance x in      | lente : $O(n)$     | rapide : $O(1)$            |
| Opérations d'ensembles | non                | oui (  & - ^)              |
| Éléments               | quelconques        | <b>hachables</b> seulement |

## Comment choisir ?

- ♦ **list** si l'**ordre** compte, si l'on accède **par indice**, ou si l'on veut **garder les doublons**.
- ♦ **set** si l'on veut des éléments **uniques**, tester souvent l'**appartenance**, ou **combiner** des ensembles.
- ♦ Conversion : **set(L)** pour dédoublonner ; **list(s)** pour retrouver ordre et indices.

109





# Sommaire

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Avant-propos

Environnement  
Python

Outils de  
contrôle de flux

Les collections  
de données

Généralités

Les listes

Les tuples

Les ensembles

Les dictionnaires

Les fonctions

Les modules

Fichiers et  
entrées/sorties

De C à Python : une transition

Avant-propos

Environnement Python

Outils de contrôle de flux

**Les collections de données**

Généralités

Les listes

Les tuples

Les ensembles

Les dictionnaires

Les fonctions

Les modules

Fichiers et entrées/sorties

80

109

# Dictionnaires : définition

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Avant-propos

Environnement  
Python

Outils de  
contrôle de flux

Les collections  
de données

Généralités

Les listes

Les tuples

Les ensembles

Les dictionnaires

Les fonctions

Les modules

Fichiers et  
entrées/sorties

♦ Un dictionnaire associe à chaque **clé** une **valeur**, entre accolades.

## Remarque

Les valeurs sont de n'importe quel type et peuvent se répéter ; les clés doivent être **immuables** (chaîne, nombre, tuple) et **uniques**.

```
>>> animal = {}  
>>> animal["nom"] = "girafe"  
>>> animal["taille"] = 5.0  
>>> print(animal)  
{'nom': 'girafe', 'taille': 5.0}
```

# Dictionnaires : accès et opérations

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Avant-propos

Environnement  
Python

Outils de  
contrôle de flux

Les collections  
de données

Généralités

Les listes

Les tuples

Les ensembles

Les dictionnaires

82

Les fonctions

Les modules

Fichiers et  
entrées/sorties

```
>>> ouvrier = {"nom": "Mohammed", "prenom": "Jellouli", "age": 48}
>>> ouvrier["nom"]           # 'Mohammed'
>>> len(ouvrier)             # 3
```

```
>>> ouvrier["ville"] = "Mohammedia"    # ajout (cle absente)
>>> ouvrier["age"] = 49                  # modification (cle presente)
>>> del ouvrier["prenom"]                # suppression d'une entree
>>> ouvrier.clear()                      # vide le dictionnaire
```

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Avant-propos

Environnement  
Python

Outils de  
contrôle de flux

Les collections  
de données

Généralités

Les listes

Les tuples

Les ensembles

Les dictionnaires

83

Les fonctions

Les modules

Fichiers et  
entrées/sorties

| Méthode                    | Description                          |
|----------------------------|--------------------------------------|
| <code>keys()</code>        | renvoie les clés                     |
| <code>values()</code>      | renvoie les valeurs                  |
| <code>items()</code>       | renvoie les paires (clé, valeur)     |
| <code>get(clé, d)</code>   | valeur de la clé, ou d si absente    |
| <code>pop(clé)</code>      | retire l'entrée et renvoie sa valeur |
| <code>update(autre)</code> | fusionne un autre dictionnaire       |

# Les dictionnaires : à quoi servent-ils ?

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Avant-propos

Environnement  
Python

Outils de  
contrôle de flux

Les collections  
de données

Généralités

Les listes

Les tuples

Les ensembles

Les dictionnaires

84

Les fonctions

Les modules

Fichiers et  
entrées/sorties

- ◆ Un dictionnaire associe une **clé** à une **valeur** : on accède aux données **par nom**, pas par position.
  - ✧ **Clés uniques** : une clé n'apparaît qu'une fois ; la réaffecter **écrase** l'ancienne valeur.
  - ✧ **Accès direct et rapide** par clé :  $O(1)$  (hachage), sans parcourir la structure.
  - ✧ **Clés hachables** (immuables : `str`, nombre, `tuple`) ; les **valeurs** sont quelconques.
  - ✧ L'**ordre d'insertion** est préservé (Python 3.7+).
- ◆ Usages phares : **table associative**, **comptage**/fréquences, **regroupement**, **mémoïsation**, données **structurées** (config, JSON).

## Règle simple

**Liste** = accès par *position* (indice).    **Dict** = accès par *clé* (nom).

# Utilité (1) : table associative et unicité des clés

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Avant-propos

Environnement  
Python

Outils de  
contrôle de flux

Les collections  
de données

Généralités

Les listes

Les tuples

Les ensembles

Les dictionnaires

Les fonctions

Les modules

Fichiers et  
entrées/sorties

85

```
>>> # une fiche : acces par cle (pas par position)
>>> etudiant = {"nom": "Sara", "age": 20, "ville": "Rabat"}
>>> etudiant["nom"]
'Sara'
>>> etudiant.get("email", "inconnu")    # valeur par default si absente
'inconnu'

>>> # cleS UNIQUES : reassigner une cle ECRASE l'ancienne valeur
>>> notes = {"Ali": 12}
>>> notes["Ali"] = 15
>>> notes
{'Ali': 15}
>>> dict([("x", 1), ("x", 2)])    # la derniere valeur l'emporte
{'x': 2}
```

## Utilité (2) : compter et regrouper

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Avant-propos

Environnement  
Python

Outils de  
contrôle de flux

Les collections  
de données

Généralités

Les listes

Les tuples

Les ensembles

Les dictionnaires

86

Les fonctions

Les modules

Fichiers et  
entrées/sorties

```
>>> # compter les occurrences (frequences)
>>> freq = {}
>>> for mot in "le chat le chien le chat".split():
...     freq[mot] = freq.get(mot, 0) + 1
>>> freq
{'le': 3, 'chat': 2, 'chien': 1}

>>> # regrouper par cle (etudiants par ville)
>>> data = [("Ali", "Rabat"), ("Sara", "Sale"), ("Omar", "Rabat")]
>>> par_ville = {}
>>> for nom, ville in data:
...     par_ville.setdefault(ville, []).append(nom)
>>> par_ville
{'Rabat': ['Ali', 'Omar'], 'Sale': ['Sara']}
```

♦ collections.Counter et defaultdict font cela en une ligne.

# Utilité (3) : données structurées et parcours

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Avant-propos

Environnement  
Python

Outils de  
contrôle de flux

Les collections  
de données

Généralités

Les listes

Les tuples

Les ensembles

Les dictionnaires

87

Les fonctions

Les modules

Fichiers et  
entrées/sorties

```
>>> # enregistrement accessible par nom de champ
>>> config = {"largeur": 1920, "hauteur": 1080, "plein_ecran": True}
>>> for cle, valeur in config.items():
...     print(cle, "=", valeur)
largeur = 1920
hauteur = 1080
plein_ecran = True

>>> cours = {"M351": {"nom": "TDM", "credits": 5}} # dict imbriquée (JSON)
>>> cours["M351"]["credits"]
5
>>> {n: n**2 for n in range(1, 5)} # comprehension de dict
{1: 1, 2: 4, 3: 9, 4: 16}
```

## À noter

in teste les clés : "M351" in cours → True. Le dict imbriqué modélise naturellement des données de type JSON.





# Sommaire

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Avant-propos

Environnement  
Python

Outils de  
contrôle de flux

Les collections  
de données

Les fonctions  
Définition et  
utilisation

Les modules

Fichiers et  
entrées/sorties

De C à Python : une transition

Avant-propos

Environnement Python

Outils de contrôle de flux

Les collections de données

**Les fonctions**  
Définition et utilisation

Les modules

Fichiers et entrées/sorties

88

109

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Avant-propos

Environnement  
Python

Outils de  
contrôle de flux

Les collections  
de données

Les fonctions  
Définition et  
utilisation

Les modules

Fichiers et  
entrées/sorties

89

109

◆ Une fonction améliore :

✧ la **modularité** : découper le code en blocs logiques lisibles ;

✧ la **réutilisation** : réaliser plusieurs fois la même opération.

◆ **Python** fournit des fonctions intégrées (`range()`, `len()`, `print()`, ...) et permet de définir les siennes.

# Définir une fonction

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Avant-propos

Environnement  
Python

Outils de  
contrôle de flux

Les collections  
de données

Les fonctions  
Définition et  
utilisation

Les modules

Fichiers et  
entrées/sorties

♦ Mot-clé `def` pour définir, `return` pour renvoyer une valeur.

```
>>> def hello():  
...     print("Bonjour depuis la fonction")  
>>> hello()  
Bonjour depuis la fonction  
  
>>> def mes_infos(nom, age):  
...     print("Nom :", nom, " Age :", age)  
>>> mes_infos(nom="Mustapha", age=50)  
Nom : Mustapha Age : 50
```

# Renvoyer plusieurs valeurs et fonctions lambda

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Avant-propos

Environnement  
Python

Outils de  
contrôle de flux

Les collections  
de données

Les fonctions  
Définition et  
utilisation

Les modules

Fichiers et  
entrées/sorties

91

```
>>> def car_cub_quad(x):  
...     return x**2, x**3, x**4      # renvoie un tuple  
>>> car_cub_quad(5)                 # (25, 125, 625)
```

♦ Une fonction **anonyme** se définit avec `lambda` (une seule expression) :

```
>>> som = lambda a, b: a + b  
>>> som(10, 20)                # 30  
>>> puiss = lambda x, y: x ** y  
>>> puiss(4, -2)                # 0.0625
```

# Les fonctions : ce qui change par rapport au C

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Avant-propos  
Environnement  
Python

Outils de  
contrôle de flux  
Les collections  
de données

Les fonctions  
Définition et  
utilisation

Les modules  
Fichiers et  
entrées/sorties

92

109

## En C

```
int carre(int x) {
    return x * x;
}
```

## En Python

```
def carre(x):
    return x * x
```

- ◆ **Ni type de retour ni type de paramètre** : typage dynamique.
- ◆ Pas de **prototype** ni d'en-tête .h, pas de déclaration en avant.
- ◆ Une fonction peut : renvoyer **plusieurs valeurs**, avoir des arguments **par défaut/nommés**, être **passée en argument**, être **imbriquée**.
- ◆ Documentation intégrée par une **docstring** ("\" . . . \").

# Spécificité (1) : des arguments souples

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Avant-propos

Environnement  
Python

Outils de  
contrôle de flux

Les collections  
de données

Les fonctions  
Définition et  
utilisation

Les modules

Fichiers et  
entrées/sorties

93

```
>>> def salut(nom, message="Bonjour"):      # argument par DEFAULT
...     return f"{message}, {nom} !"
>>> salut("Sara")
'Bonjour, Sara !'
>>> salut("Ali", message="Salut")          # argument NOMME (ordre libre)
'Salut, Ali !'

>>> def somme(*nombres):                    # nombre VARIABLE d'arguments -> tuple
...     return sum(nombres)
>>> somme(1, 2, 3, 4)
10
>>> def fiche(**infos):                    # arguments nommes -> dictionnaire
...     return infos
>>> fiche(nom="Omar", age=21)
{'nom': 'Omar', 'age': 21}
```

◆ Défaut, nommés, \*args, \*\*kwargs : inexistants ou pénibles en C (stdarg.h).

# Spécificité (2) : la fonction est un objet

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Avant-propos

Environnement  
Python

Outils de  
contrôle de flux

Les collections  
de données

Les fonctions  
Définition et  
utilisation

Les modules

Fichiers et  
entrées/sorties

94

```
>>> def carre(x): return x * x
>>> f = carre                                # une fonction s'affecte a une variable
>>> f(5)
25
>>> sorted(["banane", "kiwi", "fig"], key=len)    # passee en ARGUMENT
['fig', 'kiwi', 'banane']
>>> list(map(lambda x: x * 2, [1, 2, 3]))          # lambda : fonction anonyme
[2, 4, 6]
>>> ops = {"+": lambda a, b: a + b,               # stockee dans un dict
...        "-": lambda a, b: a - b}              # (table de dispatch)
>>> ops["+"](10, 5)
15
```

♦ **Objet de première classe** : affectée, passée, renvoyée, stockée. En C : seulement des **pointeurs de fonction**, moins souples.

# Spécificité (3) : fonctions imbriquées et closures

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Avant-propos

Environnement  
Python

Outils de  
contrôle de flux

Les collections  
de données

Les fonctions  
Définition et  
utilisation

Les modules

Fichiers et  
entrées/sorties

95

- ◆ Une fonction peut être définie **dans** une autre, **capturer** ses variables, et être **renvoyée**.

```
>>> def compteur():
...     n = 0
...     def incrementer():
...         nonlocal n           # modifie la variable de la fonction englobante
...         n += 1
...         return n
...     return incrementer      # renvoie une FONCTION (closure)
>>> c = compteur()
>>> c(), c(), c()
(1, 2, 3)
```

- ◆ c « se souvient » de son n propre. Impossible en C standard (ni fonctions imbriquées, ni closures).



# Pièges venant du C

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition  
Avant-propos  
Environnement  
Python  
Outils de  
contrôle de flux  
Les collections  
de données  
Les fonctions  
Définition et  
utilisation  
Les modules  
Fichiers et  
entrées/sorties

96

109

## Objets mutables = passés par référence

```
>>> def ajoute(liste):
...     liste.append(0)      # modifie
...                         # l'objet
...     appelle
>>> L = [1, 2]
>>> ajoute(L)
>>> L
[1, 2, 0]
```

## Défaut mutable = piège

```
>>> def f(x, acc=[]):
...     acc.append(x)      # acc est
...     return acc         # PARTAGE !
>>> f(1)
[1]
>>> f(2)
[1, 2]
```

## Règles

Une list/dict passée en argument **peut être modifiée** (référence d'objet). Et un argument par défaut mutable est créé **une seule fois** et partagé : utilisez `def f(x, acc=None): ... if acc is None: acc = []`.



# Sommaire

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Avant-propos

Environnement  
Python

Outils de  
contrôle de flux

Les collections  
de données

Les fonctions

Les modules

Importer et créer un  
module

Fichiers et  
entrées/sorties

De C à Python : une transition

Avant-propos

Environnement Python

Outils de contrôle de flux

Les collections de données

Les fonctions

**Les modules**

Importer et créer un module

Fichiers et entrées/sorties

97

109

# Définition et modules courants

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Avant-propos

Environnement  
Python

Outils de  
contrôle de flux

Les collections  
de données

Les fonctions

Les modules

Importer et créer un  
module

Fichiers et  
entrées/sorties

98

- ◆ Un **module** est un fichier `.py` regroupant fonctions et variables réutilisables autour d'un thème.
- ◆ Quelques modules de la bibliothèque standard :
  - ✧ **math** : fonctions et constantes mathématiques ;
  - ✧ **os** : dialogue avec le système d'exploitation ;
  - ✧ **random** : génération de nombres aléatoires ;
  - ✧ **time** : gestion du temps ; **urllib** : accès réseau.

109

# Importer un module

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Avant-propos

Environnement  
Python

Outils de  
contrôle de flux

Les collections  
de données

Les fonctions

Les modules

Importer et créer un  
module

Fichiers et  
entrées/sorties

```
import math                # tout le module
math.sqrt(16)              # 4.0

from math import pi, sqrt  # quelques elements
sqrt(pi)

import numpy as np         # avec un alias
```

♦ `from module import *` importe tout (à éviter : risque de collisions de noms).

# Créer son propre module

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Avant-propos

Environnement  
Python

Outils de  
contrôle de flux

Les collections  
de données

Les fonctions

Les modules

Importer et créer un  
module

Fichiers et  
entrées/sorties

- ♦ On écrit ses fonctions dans un fichier, puis on l'importe comme un module standard.

## calc.py

```
def add(x, y):
    return x + y

def sub(x, y):
    return x - y
```

## Utilisation

```
>>> import calc as op
>>> op.add(10, 20)
30
>>> op.sub(20, 5)
15
```



# Sommaire

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Avant-propos

Environnement  
Python

Outils de  
contrôle de flux

Les collections  
de données

Les fonctions

Les modules

Fichiers et  
entrées/sorties

Entrées/sorties  
standard

Les fichiers

Contact

101

De C à Python : une transition

Avant-propos

Environnement Python

Outils de contrôle de flux

Les collections de données

Les fonctions

Les modules

**Fichiers et entrées/sorties**

Entrées/sorties standard

Les fichiers

Contact

109

# print() et input()

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Avant-propos

Environnement  
Python

Outils de  
contrôle de flux

Les collections  
de données

Les fonctions

Les modules

Fichiers et  
entrées/sorties

Entrées/sorties  
standard

Les fichiers  
Contact

♦ `print()` écrit sur la sortie standard ; `input()` lit une ligne au clavier et renvoie **toujours** une chaîne.

```
>>> val = input("Entrez un entier : ")
>>> type(val)           # <class 'str'>
>>> val = int(val)      # conversion : reaffectation obligatoire
>>> type(val)           # <class 'int'>

>>> n = int(input("Entrez un entier : "))    # ou en une ligne
```

♦ `int(val)` ne modifie pas `val` : il faut récupérer la valeur convertie.



# Sommaire

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Avant-propos

Environnement  
Python

Outils de  
contrôle de flux

Les collections  
de données

Les fonctions

Les modules

Fichiers et  
entrées/sorties

Entrées/sorties  
standard

**Les fichiers**

Contact

De C à Python : une transition

Avant-propos

Environnement Python

Outils de contrôle de flux

Les collections de données

Les fonctions

Les modules

**Fichiers et entrées/sorties**

Entrées/sorties standard

Les fichiers

Contact

103

109



# Ouvrir un fichier

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Avant-propos  
Environnement  
Python

Outils de  
contrôle de flux

Les collections  
de données

Les fonctions

Les modules

Fichiers et  
entrées/sorties

Entrées/sorties  
standard

Les fichiers

Contact

- ♦ Une opération sur un fichier suit l'ordre : **ouvrir** → **lire/écrire** → **fermer**.
- ♦ Le gestionnaire de contexte **with** ferme le fichier automatiquement : c'est la forme recommandée.

```
# Forme classique  
fid = open("Etudiants.txt", "r", encoding="utf-8")  
contenu = fid.read()  
fid.close()
```

```
# Gestionnaire de contexte (recommande)  
with open("Etudiants.txt", "r", encoding="utf-8") as fid:  
    contenu = fid.read()
```

# Modes d'ouverture

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Avant-propos

Environnement  
Python

Outils de  
contrôle de flux

Les collections  
de données

Les fonctions

Les modules

Fichiers et  
entrées/sorties

Entrées/sorties  
standard

Les fichiers

Contact

105

109

| Mode | Rôle                                              |
|------|---------------------------------------------------|
| "r"  | lecture (mode par défaut)                         |
| "w"  | écriture ; crée ou <b>écrase</b> le fichier       |
| "a"  | ajout en fin de fichier                           |
| "r+" | lecture et écriture                               |
| "b"  | mode binaire, combiné aux autres (ex. "rb", "wb") |

- ♦ Correspondance directe avec le "r"/"w"/"a" de fopen en C.
- ♦ En mode **texte**, préciser encoding="utf-8" pour gérer correctement les accents.

# Lire un fichier

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Avant-propos

Environnement  
Python

Outils de  
contrôle de flux

Les collections  
de données

Les fonctions

Les modules

Fichiers et  
entrées/sorties

Entrées/sorties  
standard

Les fichiers

Contact

```
# 1) tout d'un coup -> une seule chaine
with open("notes.txt", encoding="utf-8") as f:
    contenu = f.read()

# 2) ligne par ligne (RECOMMANDE : econome en memoire )
with open("notes.txt", encoding="utf-8") as f:
    for ligne in f:
        print(ligne.strip())    # strip() retire le '\n' de fin

# 3) toutes les lignes dans une liste
with open("notes.txt", encoding="utf-8") as f:
    lignes = f.readlines()      # ['12\n', '15\n', '18\n']
```

# Écrire dans un fichier

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Avant-propos

Environnement  
Python

Outils de  
contrôle de flux

Les collections  
de données

Les fonctions

Les modules

Fichiers et  
entrées/sorties

Entrées/sorties  
standard

Les fichiers

Contact

107

```
# mode "w" : cree ou ECRASE le fichier
with open("sortie.txt", "w", encoding="utf-8") as f:
    f.write("Licence IRM\n")          # write n'ajoute PAS de \n
    f.write("Module M351\n")
    print("Ligne via print", file=f)  # print ajoute le \n

# mode "a" : AJOUTE en fin, sans effacer l'existant
with open("sortie.txt", "a", encoding="utf-8") as f:
    f.write("Une ligne de plus\n")
```

♦ `write` n'ajoute pas de saut de ligne; `print(..., file=f)` en ajoute un.

109

# Cas concret : lire et exploiter un fichier

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Avant-propos

Environnement  
Python

Outils de  
contrôle de flux

Les collections  
de données

Les fonctions

Les modules

Fichiers et  
entrées/sorties

Entrées/sorties  
standard

Les fichiers

Contact

```
# fichier "etudiants.txt" :   Sara;18   puis   Ali;15
total = 0
with open("etudiants.txt", encoding="utf-8") as f:
    for ligne in f:
        nom, note = ligne.strip().split(";")
        note = int(note)           # le fichier ne contient que du TEXTE
        total += note
    print(nom, "->", note)
```

## Bonnes pratiques

Toujours with (fermeture garantie)

- `encoding="utf-8"`
- itérer ligne par ligne
- **convertir** le texte lu (`int`, `float`) avant tout calcul.



# Feedback

## Contact Information

Apprendre à coder  
avec Python

Prof. Abdellah Adib

De C à Python :  
une transition

Avant-propos

Environnement  
Python

Outils de  
contrôle de flux

Les collections  
de données

Les fonctions

Les modules

Fichiers et  
entrées/sorties

Entrées/sorties  
standard

Les fichiers

Contact

109

109

Vos retours m'intéressent (commentaires, suggestions, signalements de bogues).  
Vous pouvez me joindre aux coordonnées ci-dessous.

Prof. Abdellah Adib

`abdellah.adib@fstm.ac.ma`

Département Informatique

Merci de votre attention!

